

Article

RGISQL: Integrating Refined Grammatical Information into Relational Graph Neural Network for Text-to-SQL Task

Shuiyan Li ¹, Yaozhen He ² , Longhao Ao ² and Rongzhi Qi ^{2,3,*} ¹ School of Mathematics, Hohai University, Nanjing 211100, China; lsy@hhu.edu.cn² College of Computer Science and Software Engineering, Hohai University, Nanjing 211100, China³ Key Laboratory of Water Big Data Technology of Ministry of Water Resources, Hohai University, Nanjing 211100, China

* Correspondence: rzqi@hhu.edu.cn

Abstract: The text-to-SQL task aims to convert natural language questions into corresponding SQL queries based on a given database schema. Previous models that rely on graph neural networks often struggle to accurately capture the complex grammatical relationships present in these questions, leading to poor performance when generating queries for longer requests. To address these challenges, we propose RGISQL, which integrates refined grammatical information extracted from the question and employs segmentation processing to effectively manage long queries. Additionally, RGISQL minimizes the complexity of edge embeddings by reducing the coupling within graph neural networks. By utilizing grammatical dependency trees, RGISQL is better equipped to capture the inherent structure and grammatical rules of questions. This refined grammatical information offers additional contextual and semantic cues for the model, thereby enhancing both its generalizability and interpretability. Furthermore, we dynamically assess the importance of different edges based on the graph structure, which helps reduce the coupling of edge embeddings and further improves the model's performance. Multiple sets of experiments conducted on the Spider and Spider-Syn datasets demonstrate that RGISQL outperforms other baselines, achieving the best results in both datasets.

Keywords: text-to-SQL task; grammatical dependency tree; segmentation processing; reducing coupling



Citation: Li, S.; He, Y.; Ao, L.; Qi, R. RGISQL: Integrating Refined Grammatical Information into Relational Graph Neural Network for Text-to-SQL Task. *Appl. Sci.* **2024**, *14*, 10359. <https://doi.org/10.3390/app142210359>

Academic Editor: Christos Bouras

Received: 11 October 2024

Revised: 5 November 2024

Accepted: 8 November 2024

Published: 11 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The goal of the text-to-SQL task is to translate natural language questions into their corresponding SQL queries. Querying relational databases, which are knowledge bases that require specific SQL statements, poses a challenge for non-experts. Consequently, the ability to query databases by using natural language has attracted many users unfamiliar with SQL, enabling them to access large databases. Many neural methods have been proposed to address this task [1,2].

Large datasets containing natural language questions and their corresponding SQL queries have significantly advanced the development of this field. Compared with previous datasets, multi-domain datasets containing complex information, such as Spider [3] and Spider-Syn [4], present new challenges. Most queries are based on a multi-table database schema, which contains multiple levels of nesting.

Since the proposal of large-scale text-to-SQL datasets, various types of relationships have been incorporated into this task. In models that incorporate relational structures, it is particularly important to encode both the question and the elements of the database schema, as well as to understand the various relationships among these heterogeneous inputs. Previous studies have employed node-centric graph neural networks (GNNs) [5] to aggregate information from neighboring nodes. For example, GNNSQL utilizes a relational graph convolutional network (RGCN) [6] to address the various types of edges within the

schema elements. However, these edge features are obtained directly from a fixed-size parameter matrix, which does not aid in understanding contextual information, particularly the topological structure of the edges. Although RAT-SQL [7] introduces several useful meta-paths, it represents both one-hop and multi-hop relationships in the same way within the graph. In LGESQL [8], the incorporation of a line graph enables a distinction between one-hop and multi-hop relationships while also taking into account the topological structure of the edges, significantly improving performance on these tasks. RASAT [9] incorporates the dependency structure of natural language questions and introduces multi-head relation-aware attention into the T5 model.

Although graph neural networks have been widely applied in text-to-SQL tasks, previous SQL methods have not fully leveraged the nuanced grammatical information present in sentences, leading to limitations in syntactic parsing and semantic understanding. As a result, misunderstandings of SQL query intent arose during the generation process, resulting in the creation of incorrect queries. For instance, as illustrated in Figure 1, if the method emphasizes only the main structure of the sentence, it cannot guarantee that the term “age” is included in the final generated SQL query. Furthermore, the model tends to perform inadequately when handling long and complex queries, demonstrating weak generalization capability.

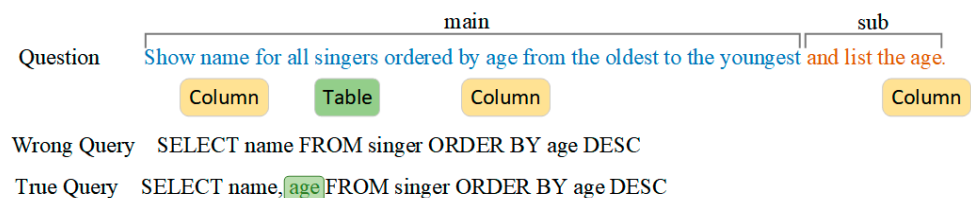


Figure 1. A simple example of parsing natural language questions and generating SQL queries.

In this paper, we propose RGISQL, which integrates refined grammatical dependencies among question tokens, enabling the model to focus more intently on the crucial grammatical information inherent in the questions. To further enhance the model’s performance on long queries, we implement a segmentation approach. Lastly, to boost the model’s generalization capability, we reduce the complexity of edge embeddings by minimizing coupling. RGISQL also integrates recursive decoding with the ASTormer decoder during the decoding process. The experimental results indicate that utilizing a pre-trained model tailored for the preprocessing stage enables us to achieve an exact set match accuracy of 73.6% and an execution accuracy of 77.3% on the Spider dataset. This demonstrates that the SQL queries generated by RGISQL are more accurate. Additionally, we achieve an accuracy of 52.6% on Spider-Syn, highlighting a significant improvement in the model’s generalization ability compared with the baseline. Furthermore, we demonstrate the effectiveness of our method through comprehensive ablation experiments.

The primary contributions of this paper are summarized as follows:

1. For text-to-SQL tasks, we propose RGISQL, which incorporates grammatical information and utilizes segmentation processing to effectively handle long queries. It represents grammatical information in natural language questions by using a grammatical dependency tree, refining the relationships between different words in the question. This approach enhances the model’s understanding of grammatical information and improves its generalizability.
2. We introduce dynamic edge attention pooling in the Relational Graph Attention Network (RGAT), which reduces the over-coupling of edge embeddings during training. Additionally, we incorporate recursive decoding with the ASTormer decoder, thereby enhancing the model’s expressiveness and performance.
3. We conduct extensive experiments on the Spider and Spider-Syn datasets to evaluate RGISQL, and the results indicate that RGISQL outperforms baseline methods. More-

over, we perform ablation studies across multiple dimensions, demonstrating the effectiveness of the proposed approach.

The remainder of this paper is organized as follows: Section 2 reviews the literature related to the text-to-SQL task. Section 3 presents the background of RGISQL. Section 4 introduces our approach. Section 5 discusses the experimental setup and results. Section 6 concludes with recommendations for future work.

2. Related Studies

Text-to-SQL semantic analysis essentially involves translating natural language into a logical format [10,11]. Early approaches often utilized sketch-based slot-filling methods, employing various modules to predict the corresponding components of SQL queries. These methods divided the SQL generation task into multiple independent sketches and used different classifiers to predict the respective parts, such as SQLNet [12], SyntaxSQLNet [13], and RYANSQL [14]. However, most of these methods only handled simple queries and could not generate correct SQL for complex datasets (e.g., Spider).

In the context of multiple tables and complex SQL settings, utilizing graph structures to encode various intricate relationships has emerged as a significant trend in the text-to-SQL task [15–17]. Significant efforts have been made to enhance both encoders and decoders. For instance, Global-GNN [18] represents complex database schemas as graphs. RAT-SQL [7] introduces schema encoding and schema linking, assigning a relationship to every pair of input items. Scholak et al. [19] proposed DuoRAT, which is based on RAT-SQL. Instead of building increasingly complex encoding structures, the DuoRAT model primarily focuses on simplifying certain aspects of the RAT-SQL model. ShadowGNN [20] first utilizes a graph mapping network to abstract the semantic information from natural language and database schemas, thereby obtaining a simplified structure. It then employs the RAT framework to achieve comprehensive encoding, mitigating the impact of domain-specific information and enhancing the model’s cross-domain capabilities.

LGESQL [8] integrates relational structures by combining graph neural networks, while RASAT [9] introduces relational structures in T5 through attention mechanisms, thereby better utilizing the parameters of pre-trained models. However, both approaches treat the importance of various grammatical dependency relationships in natural language questions equally, without fully considering the significance of fine-grained grammatical information within sentences. This limitation affects syntactic parsing and semantic understanding. Therefore, the proposed approach refines grammatical dependencies by incorporating a grammatical dependency tree, which enhances the understanding of natural language questions.

Additionally, both methods directly acquire grammatical dependency relationships from the original natural language questions. However, when dealing with long queries, the obtained relationships become too complex, resulting in suboptimal performance. To address this issue, we introduce segmentation processing, dividing long queries based on their structure and syntax, which effectively tackles the challenges posed by long queries.

3. Background

In this section, we first introduce the problem definition and the Graph Attention Network (GAT) [21], which serves as the foundation for RGISQL. Next, we present the relation-aware transformer, which employs an attention mechanism that incorporates relational information when computing attention weights.

3.1. Problem Definition

Given a natural language question and the corresponding database schema $S = T \cup C$, the goal is to generate an SQL query Y . Let $Q = (q_1, q_2, \dots, q_{|Q|})$ be a sequence of natural language tokens with a length of $|Q|$. The database schema consists of multiple tables $T = \{t_1, t_2, \dots\}$ and columns $C = \{c_1^{t_1}, c_2^{t_1}, \dots, c_1^{t_2}, c_2^{t_2}, \dots\}$. Each table t_i is composed of

several words $(t_{i1}, t_{i2}, \dots)$. Similarly, we use the word sequence $(c_{j1}^{t_i}, c_{j2}^{t_i}, \dots)$ to represent columns $c_j^{t_i}$ within t_i . Additionally, each column $c_j^{t_i}$ has a type field $c_{j0}^{t_i}$ that constrains the values of its cells (e.g., TEXT and NUMBER). In this work, the final predicted SQL query is represented as a sequence of tokens denoted by $Y = \{y_1, y_2, \dots, y_{|Y|}\}$.

3.2. Graph Attention Network

GAT [21] is a graph neural network developed for processing graph-structured data, with the aim of learning the relationships between nodes and their local structural information. As shown in Figure 2, GAT combines the principles of self-attention mechanisms with graph convolutional neural networks, effectively capturing the complex nonlinear relationships between nodes by learning the interactions between each node and its neighbors.

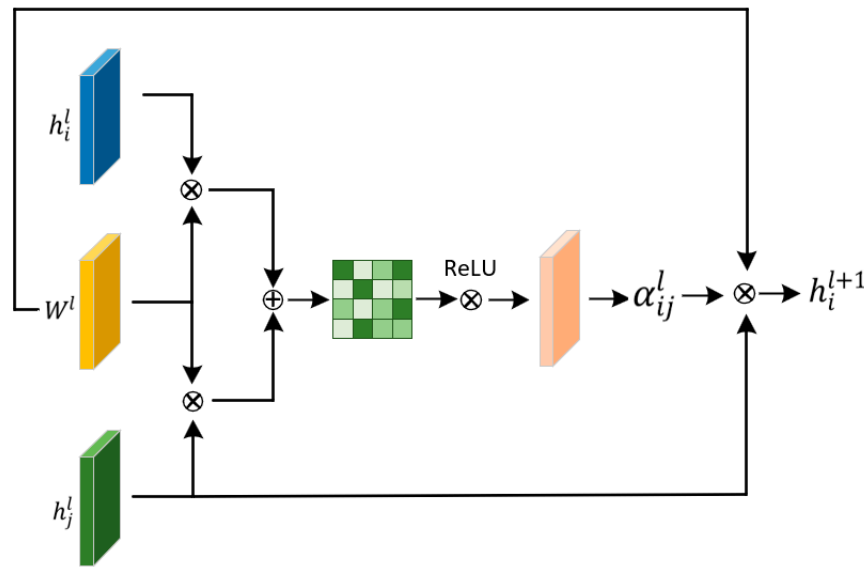


Figure 2. GAT principle data flow diagram.

The core idea of GAT is to utilize attention mechanisms to dynamically compute the weights between a node and its surrounding nodes, allowing for more effective aggregation of information from its neighbors. In each layer, GAT calculates an attention coefficient for each node, reflecting its importance in relation to its neighbors. These attention coefficients are then used to compute a weighted sum of the features from the neighboring nodes, resulting in a new representation for each node. The formula for GAT is as follows:

$$h_i^{(l+1)} = \sigma \left(\sum_{j=1}^N \alpha_{ij}^{(l)} W^{(l)} h_j^{(l)} \right), \quad (1)$$

where $h_i^{(l)}$ is the feature representation of node i in layer l , $W^{(l)}$ is the weight parameter for layer l , $\alpha_{ij}^{(l)}$ is the attention coefficient from node i to node j , and σ is the activation function, typically ReLU or LeakyReLU. The attention coefficient $\alpha_{ij}^{(l)}$ is calculated as follows:

$$\alpha_{ij}^{(l)} = \frac{\exp \left(\text{LeakyReLU} \left(a^\top \left[W^{(l)} h_i^{(l)} \parallel W^{(l)} h_j^{(l)} \right] \right) \right)}{\sum_{k \in \mathcal{N}(i)} \exp \left(\text{LeakyReLU} \left(a^\top \left[W^{(l)} h_i^{(l)} \parallel W^{(l)} h_k^{(l)} \right] \right) \right)}, \quad (2)$$

where a is the self-learned attention parameter and LeakyReLU is an activation function that enhances the nonlinear characteristics of neural networks. $\mathcal{N}(i)$ represents the set of neighboring nodes of node i , and $\parallel$ denotes the concatenation operation.

3.3. Relation-Aware Transformer

The relation-aware transformer is an enhancement of the traditional transformer model that employs relation-aware self-attention [22] by incorporating relation embeddings into the key and value entries to augment the vanilla self-attention [23]. For the input node embeddings $X = \{x_i\}_{i=1}^n$, the formula to compute the attention scores is as follows:

$$e_{ij}^{(h)} = \frac{x_i \mathbf{W}_Q^{(h)} (x_j \mathbf{W}_K^{(h)} + r_{ij,K})^\top}{\sqrt{d_z/H}}, \quad (3)$$

$$\alpha_{ij}^{(h)} = \text{softmax}_j \{e_{ij}^{(h)}\}, \quad (4)$$

where $\mathbf{W}$ is the trainable parameter matrix, h is the number of heads, $\alpha_{ij}^{(h)}$ represents the computed attention scores, and the edges from element x_i to element x_j are represented by $r_{ij,K}$ and $r_{ij,V}$. The final output $z_i^{(h)}$ is computed as shown below:

$$z_i^{(h)} = \sum_{j=1}^n \alpha_{ij}^{(h)} (x_j \mathbf{W}_V^{(h)} + r_{ij,V}), \quad (5)$$

where $r_{ij,K}$ and $r_{ij,V}$ are shared across different attention heads. For each layer of RAT, the update process can be simply represented as follows:

$$Y = \text{RAT}(X, \mathcal{R}), \quad (6)$$

where $\mathcal{R} = \{R\}_{i=1, j=1}^{|X|, |X|}$ is the relationship matrix between tokens, representing the relationship between the i -th token and the j -th token.

4. Model

The overall architecture of RGISQL adopts an encoder–decoder structure, as illustrated in Figure 3. The input for RGISQL consists of natural language questions and database schemas. Initially, the model extracts schema encodings, schema linking, question relationships, and relationships among segments, tables, and columns obtained through segmentation processing, which are represented as an interaction graph. In the input module, BERT is used to process the interaction graph, generating node embeddings X and edge embeddings R . The encoder employs a relation graph attention network (RGAT) architecture, which includes both node-centric and edge-centric graphs. During the encoder's output stage, multiple relation-aware transformer (RAT) layers integrate the updated embeddings to form a unified representation. Finally, a decoder based on an abstract syntax tree performs recursive decoding on the unified representation to generate the final SQL query.

4.1. Interaction Graph

In RGISQL, it is essential to convert the input sequence into a graph representation, where the head and tail of each relationship are tokens from the input sequence. This graph is referred to as an interaction diagram. As illustrated in Figure 4, the nodes in the graph comprise tables and columns from the database schema, entities and values mentioned in the question, as well as segments derived from the segmentation process.

First, the input is represented as a sequence X with a length of $|X|$. We assume that for each direction of a given entity pair, there exists at most one relationship. Then, the tokens in the sequence are considered vertices of a graph, which can have at most $|X|^2$ directed edges. Each edge corresponds to an entry in the adjacency matrix of the graph. In this work, the graph that represents the entire input sequence of tokens as vertices and the relationships between those tokens as edges is referred to as the interaction graph.

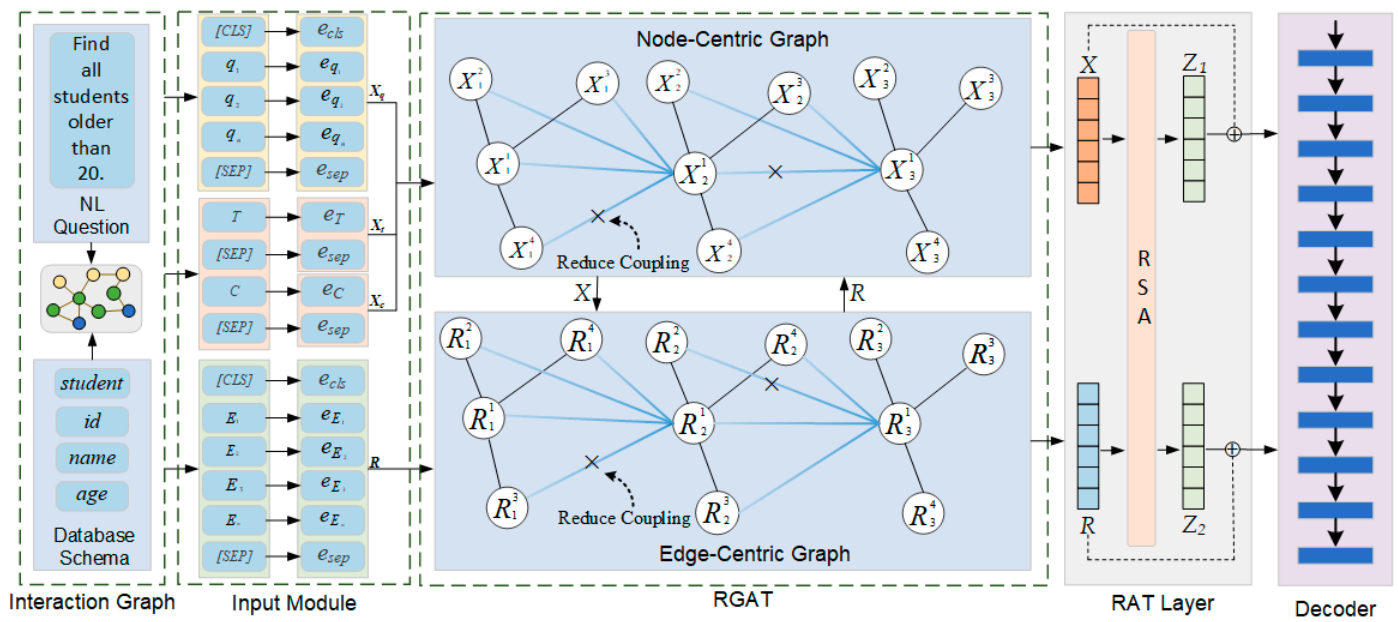


Figure 3. The overall architecture of RGISQL. X represents the node embeddings and R represents the edge embeddings. RSA stands for relation-aware self-attention mechanism.

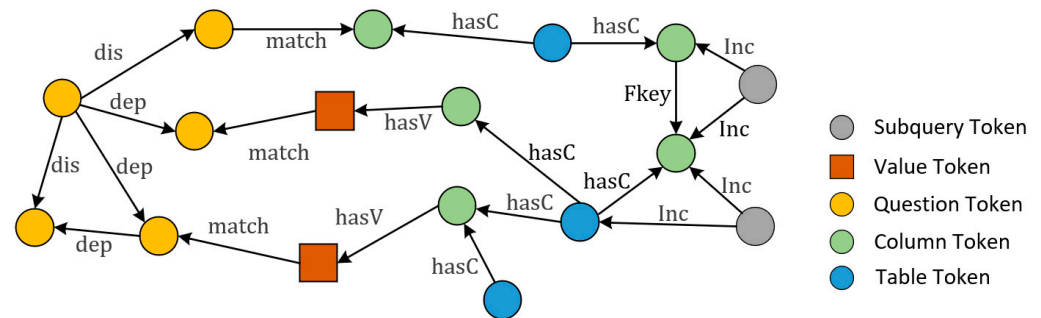


Figure 4. Input interaction diagram example.

From a generic relational perspective of the model, the inputs encompass three main types of relationships: schema encoding, schema linking, and question relations with segmentation. Important examples of each type are provided in Table 1, with question relations discussed in Section 4.2.

Table 1. Some important relational examples of each type.

Type	Head H	Tail T	Edge Label	Description
Schema encoding	$\mathcal{T}$	$\mathcal{C}$	PRIMARY-KEY	T is the primary key for H
			BELONGS-TO	T is a column in H
	$\mathcal{C}$	$\mathcal{C}$	FOREIGN-KEY	H is the foreign key for T
Schema linking	$\mathcal{Q}$	$\mathcal{T}/\mathcal{C}$	EXACT-MATCH	H is part of T, and T is a span of question
			PARTIAL-MATCH	H is part of T, and question does not contain T
Question relation	$\mathcal{Q}$	$\mathcal{Q}$	Q-Q-DIST	The distance relationship between H and T
			Q-Q-DEPENDENCY	H has a grammatical dependency with T
Segmentation	Seg	$\mathcal{T}/\mathcal{C}$	INCLUDE	Seg has T or C

4.1.1. Schema Encoding

Schema encoding is a method for identifying and classifying elements within a database schema. It involves encoding various components of the database, including tables, columns, and schema encoding relationships, according to specific rules. Schema encoding relationships refer to the connections between schema items, capturing the structural information inherent in the database schema. For example, “PRIMARY-KEY” denotes which column serves as the primary key, “BELONGS-TO” indicates which table a column is associated with, and “FOREIGN-KEY” signifies the relationship connecting a foreign key in one table to the primary key in another table.

4.1.2. Schema Linking

Schema linking relationships refer to the connections between schema items and question components, serving as a form of prior knowledge [24,25]. Due to frequent mismatches between natural language references and their corresponding names in the schema, establishing these linking relationships can be challenging. We adopt the schema linking relationships defined in RAT-SQL and identify the connections between questions and schemas based on the degree of match. Consequently, a schema linking relationship can be represented as r_{ij} , indicating the degree of match between the i -th word in the question and the j -th node name in the database schema.

4.1.3. Segmentation Processing

To manage long natural language queries, we utilize segmentation processing, which divides the lengthy query into multiple smaller segments, encodes each segment individually, and subsequently combines the information from these segments to construct the complete graph structure and feature vectors.

The primary objective of our segmentation process is to decompose a long query into several subquery segments. Initially, segmentation is conducted based on the structural features of the query, followed by a dynamic division of the long query according to its grammatical structure. An example is illustrated in Figure 5.

Long Query: Find all red cars that are available and priced under \$20,000.

Structural Division: Find all red cars that are available and priced under \$20,000.

Grammatical Division: Find all red cars that are available and priced under \$20,000.

Figure 5. The two stages of segmentation processing.

4.2. Question Relation Analysis

To improve the model’s capabilities in the syntactic parsing and semantic understanding of sentences, allowing it to generate more accurate SQL queries, we incorporate the grammatical dependencies between words in natural language questions into the graph as new relationship types. RGISQL represents the dependencies among different words in a natural language question by using a grammatical dependency tree. Initially, we classify these dependencies into two primary categories to provide a general partitioning of the relationships between various tokens. Subsequently, to highlight the significance of key components in natural language queries during model training and to enhance the model’s accuracy in understanding the grammatical meaning of sentences, we specifically designate four types of relationships: conj (coordinating conjunction), dobj (direct object), iobj (indirect object), and nsubj (the relationship between a verb and its subject).

Overall, we start by calculating the distances between different words. If the distance between two words is less than the maximum relational distance specified in the question, we set the distance to the calculated first-order distance. If it exceeds the maximum relational distance, then there is no relationship between the words in the question. In terms of grammar, as illustrated in Figure 6, we first apply grammatical constraints to divide the sentence into two parts: verb phrases and noun phrases. This process enables

us to establish the grammatical structure of the natural language query. Next, we use the Stanford Natural Language Analyze Toolkit to process the grammatical information in the questions, performing tokenization and normalization. The toolkit assigns a dependency label to each word in the sentence, indicating its grammatical relationship with other words. Then, we categorize all but four specific types of relationships into two major categories based on their antecedent and subsequent relationships. We specially mark the four refined relationships; this marking is designed to emphasize the dependency of key components in the sentence, with the aim of further clarifying the internal relationships within natural language questions. Finally, by constructing graphical tools, we introduce relationships into a heterogeneous graph. In RGAT, we use two major categories of relationships and the four refined types of relationships as edge labels, attaching these labels to the edges during graph construction. This allows RGAT to recognize and utilize these refined relationship types to better capture the grammatical structure of sentences and the dependency relationships between words.

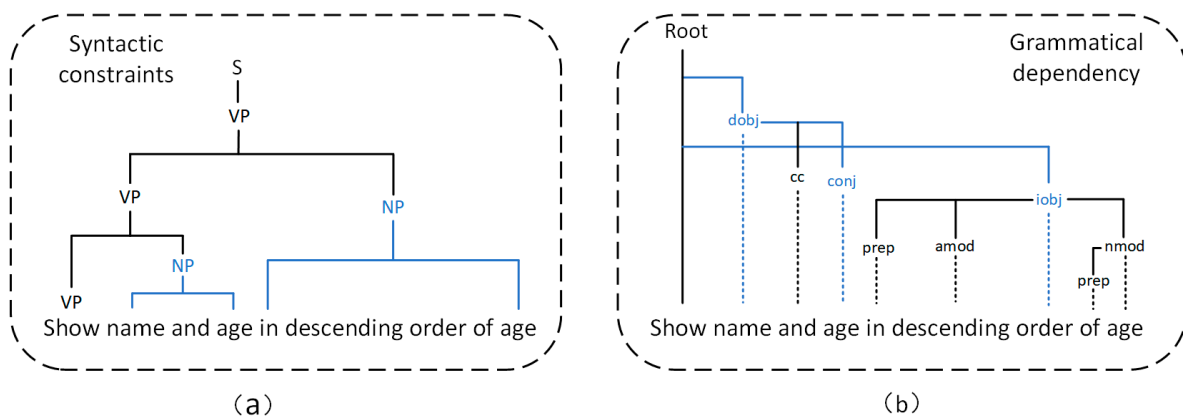


Figure 6. Results of syntactic analysis: (a) syntactic constituent analysis; (b) grammatical dependency relations.

4.3. Input Module

The input sequence is represented as a heterogeneous graph, which is then transformed into the corresponding node and relation embeddings for initialization. These representations are derived from a pre-trained language model used in this study.

Specifically, all questions and schema items are flattened into a sequence that groups together columns belonging to the same table:

$$[\text{CLS}]q_1q_2 \cdots q_{|Q|}[\text{SEP}]t_{10}t_1c_{10}^t c_1^t c_{20}^t c_2^t \cdots t_{20}t_2c_{10}^b c_1^b c_{20}^b c_2^b \cdots [\text{SEP}]seg_1seg_2seg_3seg_4 \cdots [\text{SEP}].$$

Information regarding types t_{i0} or c_{j0}^t is inserted at the beginning of each pattern item. Then, for each node i , its features are combined with the information from connected nodes in the adjacency matrix to generate node embeddings. Concurrently, the relationships between nodes are encoded as edge embeddings, reflecting the methods of connection and the significance of these relationships. This process captures the complex interactions among nodes and the information within the graph structure, integrating them into a low-dimensional embedding space for improved representation and analysis of graph data.

4.4. Relational Graph Attention Network

RGAT [26] builds upon the foundation of GAT [21] by incorporating relation-aware self-attention. In RGAT, the relational graph comprises a set of interconnected nodes, where each node represents an entity that can be linked to others, with edges denoting the relationships between them. The principles of RGAT are illustrated in Figure 7. For node A, its neighboring nodes are categorized into several groups based on different types of relationships. Taking node E as an example, let e_a represent the initial embedding of

node A, e_e represent the edge embedding from node A to neighboring node E, and e_{r_1} represent the refined grammatical relationship embedding from node A to node E. Based on this, after processing through a relation-aware self-attention mechanism, each neighboring node generates a new embedding vector that reflects the specific relationship between node A and its neighbors. Finally, all embedding vectors of different relationship types are integrated and fused through a fully connected layer to obtain the final representation of node A.

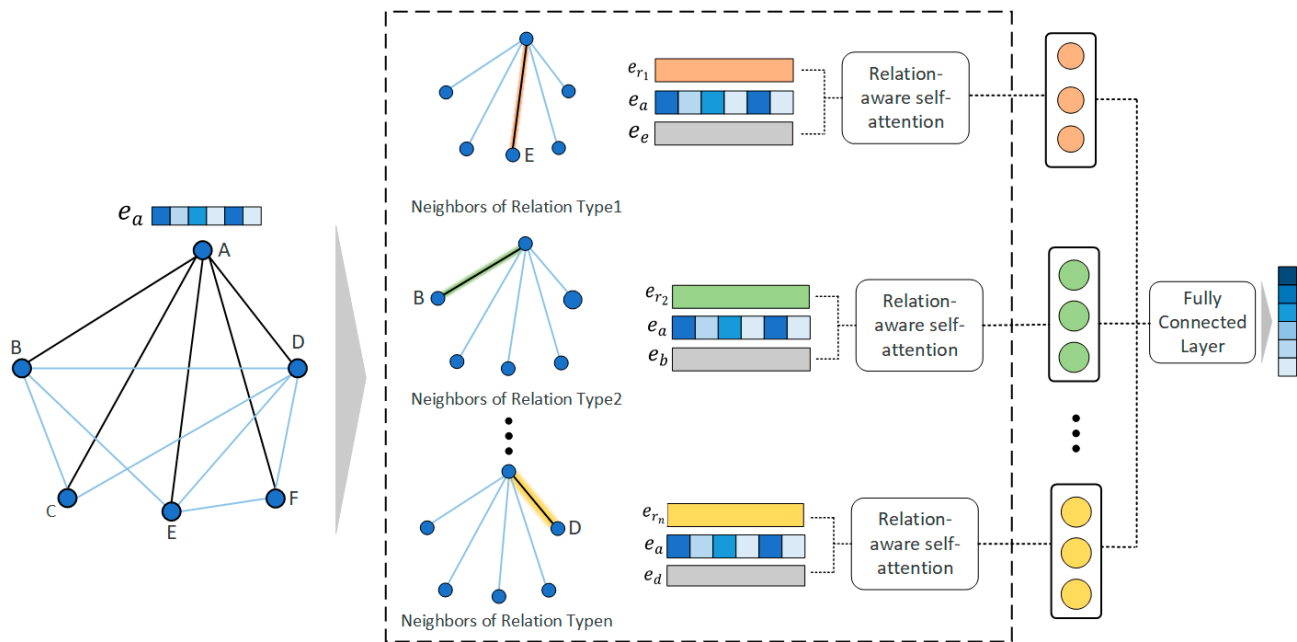


Figure 7. RGAT principle diagram.

In this paper, we employ RGAT, which comprises two primary modules: a node-centric graph module and an edge-centric graph module. The node-centric graph module represents entities as nodes, with edges indicating the relationships between them. Conversely, the edge-centric graph module treats the relationships between two entities as nodes, depicting the entities as edges in a directed graph. By combining and stacking these two graph types, our model can effectively capture and update both the nodes and their interrelationships during training while also refreshing the embeddings of both nodes and relationships.

For example, the natural language query “Find all students older than 20 years old” prompts RGISQL to first identify the database schema related to the query, including table names and column names. Then, it uses a grammatical dependency tree to parse the input’s syntactic structure and segment it, identifying key content involved in the query, such as the “students” table and the “age” field, and mapping them to the database schema.

After parsing the syntactic information, the model integrates the relationships between syntactic dependencies and the database schema into a graph structure. The graph includes nodes (such as “students” and “age”) and edges (such as the condition “age greater than 20”), generating node embeddings X and edge embeddings R. RGAT, as a relational graph encoder, is capable of capturing the connections between grammatical dependencies and the database schema, thus preparing for the next step of decoding.

Once encoding is complete, the ASTormer decoder recursively generates the SQL query step by step. It relies on the embedded information output by the encoder to construct components of the SQL statement layer by layer until the complete query is formed. The final output of the SQL statement is `SELECT * FROM student WHERE age > 20`.

4.5. Reduce Coupling

To prevent over-coupling in edge embeddings, we employ the method of dynamic edge attention pooling [27]. This approach dynamically captures the importance of different edges based on the graph's structure, thereby enhancing the representational capability and performance of the graph neural network. The process is illustrated in Figure 8.

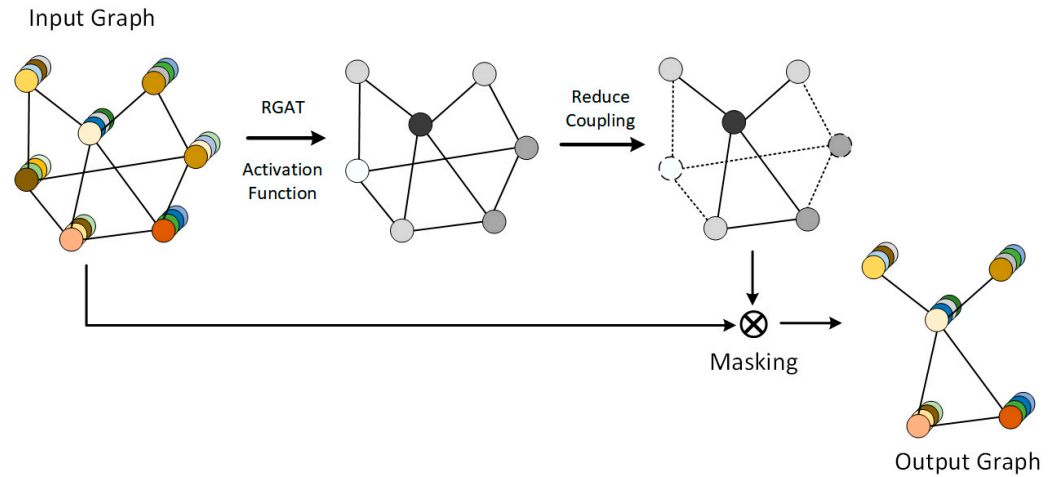


Figure 8. Over-coupling phenomenon of edge embedding, using dynamic edge attention pooling to reduce over-coupling.

The input graph is represented as $G = (V, E)$, where V denotes the set of nodes and E denotes the set of edges. For each edge, a corresponding attention weight representing the importance of the relationship between node i and node j is defined. This relationship importance is dynamically computed based on the features of the nodes, rather than being pre-defined. The formula for computing the edge attention score is described as follows:

$$e_{ij} = \text{LeakyReLU}\left(\mathbf{w}^\top [h_i || h_j]\right), \quad (7)$$

where h_i represents the set of neighboring nodes of node i , $||$ denotes the concatenation operation, $\mathbf{w}$ is the learned weight matrix, and LeakyReLU is the activation function used to enhance the model's nonlinear fitting capability. Based on the computed edge attention scores e_{ij} , the calculation of edge attention weights can be performed as follows:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})}, \quad (8)$$

where N_i represents the set of neighboring nodes of node i . Once the edge attention weights are obtained, feature aggregation can be performed on the corresponding edges as follows:

$$\tilde{h}_i = \sigma\left(\sum_{j \in N_i} \alpha_{ij} \mathbf{W} h_j + \mathbf{b}\right), \quad (9)$$

where $\mathbf{W}$ and $\mathbf{b}$ are learned parameters and σ represents the activation function. Finally, graph-level pooling is performed to obtain the final pooled graph representation as follows:

$$\mathbf{h}_{\text{graph}} = \text{AGG}\left(\left\{\tilde{h}_i \mid i \in V\right\}\right), \quad (10)$$

where AGG represents the aggregation function. By performing dynamic edge attention pooling operations, we can effectively aggregate information from various edges in the graph and assign weights based on their importance. This approach reduces the over-coupling of edge embeddings, resulting in an improved graph representation.

4.6. RAT Layer

The input for RGISQL includes natural language (NL) questions, database schemas, and question relations. RAT-SQL [7] employs a relation-aware self-attention mechanism to facilitate global reasoning for both the schema and the questions. This mechanism is used to encode the relational structures present in the given questions and database schemas, allowing the model to effectively capture their complex interactions. The principle is illustrated in Figure 9.

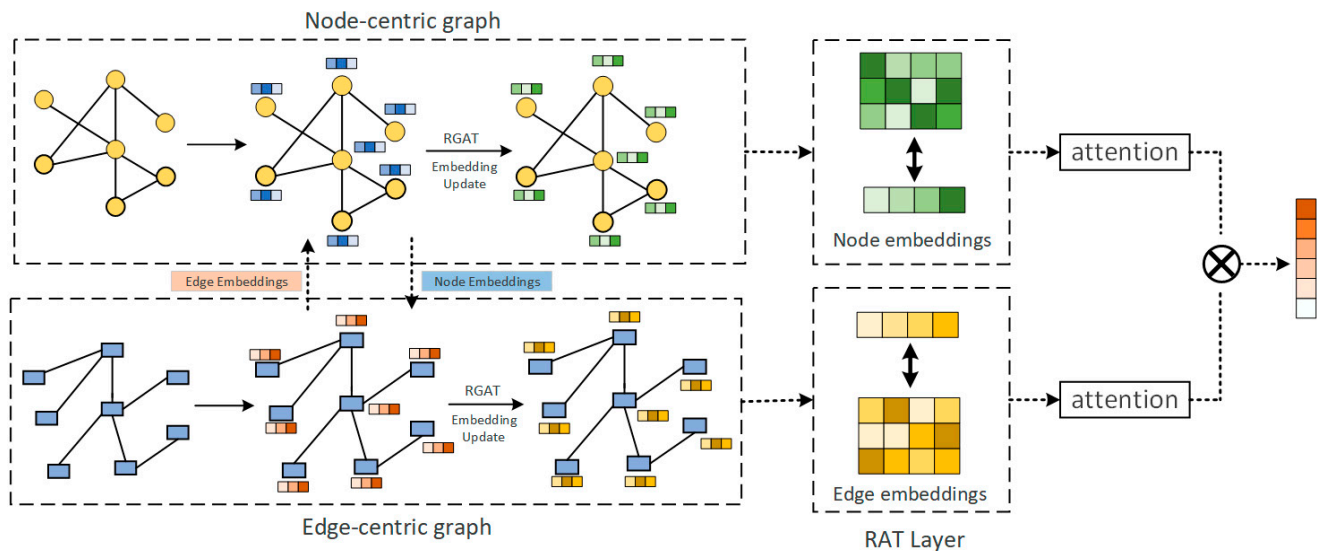


Figure 9. RAT layer embedding concatenation principle diagram.

At the input stage, the three types of input are all represented as directed graphs, and a pre-trained language model (such as BERT) is employed to obtain the embeddings for nodes and relations within the input. These embeddings are then updated by using a relational graph neural network, and RAT is utilized to achieve a unified representation of the updated embeddings.

The RAT layer employs a relation-aware self-attention mechanism to weight the node and relationship embeddings, considering the importance of different nodes and relationships. The self-attention mechanism enables the model to focus on the most critical information when generating SQL statements. The processed embedding representations are integrated into a unified representation that includes key information about the question and the database schema, providing rich context for generating SQL statements. The formula is as follows:

$$P^{(M)} = \text{RAT}\left(\text{concat}\left(X^{(N)}, R^{(N)}\right)\right), \quad (11)$$

where X is the processed node embedding, R is the processed edge embedding, M is the number of layers in the RAT layer, concat represents the operation of concatenating embeddings, and N is the number of layers in RGAT. P is the unified representation of embeddings after passing through the RAT layer, which will be used as the initial input for the decoder. This representation contains key information about the question and the database schema, enabling the decoder to generate SQL statements more accurately.

4.7. Decoder

The ASTormer decoder [28] is utilized in conjunction with a recursive decoding method. This recursive decoder leverages structural information from the graph and node feature vectors to incrementally generate the abstract syntax tree (AST) of the SQL query. This process is supported by the self-attention mechanism of the decoder. Moreover, during AST generation, the ASTormer decoder enhances the model's ability to comprehend and produce query structures and semantics. Together, the recursive decoder and the ASTormer

decoder work in tandem to efficiently and accurately convert natural language into SQL queries. The recursive decoder is responsible for mapping high-dimensional features to the fundamental structure of SQL statements, while the ASTormer decoder further optimizes and refines the SQL generation process, thus improving the model's expressiveness and overall performance.

5. Experiment

5.1. Experimental Setup

5.1.1. Dataset

The proposed model, RGISQL, was evaluated by using two public datasets: Spider and Spider-Syn. Spider is a large-scale, cross-domain text-to-SQL dataset that includes complex natural language questions and SQL queries from multiple databases. It tests the model's ability to generate accurate SQL across different database schemas, assessing its generalization performance. Spider-Syn is a variant of Spider that contains different formulations of the original questions, examining the model's ability to handle multiple linguistic expressions of the same query, thereby enhancing the model's language robustness.

5.1.2. Evaluation Metrics

In this paper, exact set match accuracy and execution accuracy were adopted as the evaluation metrics for Spider, while only execution accuracy was adopted for Spider-Syn. For exact set match accuracy, instead of simply conducting a string comparison between the predicted and gold SQL queries, each SQL query was decomposed into several clauses, where a set comparison was performed. Exact set match accuracy is given by the following formula:

$$EM = \frac{TF}{TF + NF}, \quad (12)$$

where EM represents the obtained exact set match accuracy, TF denotes the number of results that are the same as the gold SQL, and NF represents the number of results that are generated incorrectly. Execution accuracy is given by the following formula:

$$EX = \frac{TQ}{TQ + NQ}, \quad (13)$$

where EX represents the obtained execution accuracy, TQ denotes the queries that produce the same execution results as the gold SQL, and NQ represents the queries with different results.

Exact set match accuracy and execution accuracy provide a comprehensive evaluation of the effectiveness of SQL generation at both structural and semantic levels, ensuring that the model is both precise and practical in the task of SQL query generation.

5.2. Implementation

In this study, we first used Stanza [29] to tokenize and lexically process the questions, table names, and column names included in the dataset, ensuring the quality and consistency of the input data. Subsequently, basic cleaning operations were performed, including the removal of irrelevant symbols and stop words, to enhance the model's performance on the SQL generation task. Then, by using the Stanford Natural Language Processing Toolkit, we extracted detailed dependency relationship information from the questions to ensure the effective identification of important dependencies and syntactic structures, significantly enhancing the model's understanding of syntactic information. We utilized the DGL library to construct node-centric and edge-centric graphs, effectively handling different types of relationships and accurately assigning appropriate edge labels for each relationship. This approach facilitates RGAT's learning of complex relationships.

In the embedding generation phase, we employed the ELECTRA [30] model to obtain both node embeddings and edge embeddings, taking full advantage of its strengths in contextual understanding and feature representation. For the encoder selection, we adopted

RGAT as the main structure to update both edge embeddings and node embeddings, while the decoder was based on ASTormer to further enhance the model’s expressive capabilities and performance.

Hyperparameters were tuned on the validation dataset by using grid search. The specific values of the hyperparameters are listed in Table 2. In the encoder, the PLM setting was 512, the number of GNN layers was set to 8, and the dropout rate was 0.1. In the decoder, the hidden state dimension was 512, the inner layer dimension of the feedforward network was 1024, and the relation embeddings were of size 128, while the node type embeddings were of size 64. We used PyCharm for coding, and the experiments were run on two NVIDIA GeForce RTX 3090 GPUs.

Table 2. Model hyperparameter settings.

Hyperparameters	Values
PLM	512
GNN layers	8
Dropout rate	0.1
Hidden state dimension	512
Inner layer dimension	1024
Relation embedding	128
Node embedding	64
Dropout probability	0.2

5.3. Baseline Models

We conducted experiments on the Spider and Spider-Syn datasets and compared our results with several baseline models. These baseline models are based on relational or graph neural networks that focus on accurate set matching without considering the values in the Spider list; they included the following:

- RAT-SQL [7] is a relational schema encoding model.
- ShadowGNN [20] addresses schemata at both the abstract and semantic levels to reduce inter-domain discrepancies.
- LGESQL [8] is a Text-to-SQL model enhanced by line graph embedding, which classifies relations into local and non-local categories.
- RASAT [9] incorporates the dependency structure of natural language questions and introduces multi-head relation-aware attention into the T5 model.
- S²SQL [31] incorporates syntactic information into the model and decouples constraints.

5.4. Main Result

To demonstrate that RGISQL can improve text-to-SQL accuracy, it was compared with the aforementioned baselines, and the results for the Spider dataset are shown in Tables 3 and 4. The bold entries represent the optimal experimental results.

Table 3. The exact set match accuracy of different methods on the Spider dev and test sets.

Models	Dev	Test
Global-GNN [18]	52.7	47.4
IRNet [24]	64.9	55.0
RATSQL+BERT [7]	69.7	65.6
ShadowGNN+RoBERTa-large [20]	72.3	66.1
RATSQL+ELECTRA+ASTormer [28]	74.6	-
RASAT+PICARD [9]	75.3	70.9
LGESQL+ELECTRA [8]	75.1	72.0
S ² SQL+ELECTRA [31]	76.4	72.1
RGISQL+ELECTRA	77.2	73.6

Table 4. The execution accuracy of different methods on the Spider dev and test sets.

Models	Dev	Test
BRIDGE+BERT [32]	70.3	68.3
RaSaP+ELECTRA [33]	-	70.0
SmBoP+GraPPa [17]	75.0	71.1
RATSQL+GAP+NatSQL [34]	75.0	73.3
RASAT+PICARD [9]	80.5	75.5
RGISQL+ELECTRA	82.6	77.3

As shown in Table 3, RGISQL outperformed all baseline models on the Spider dataset. It is noteworthy that RGISQL+ELECTRA achieved 73.6% on the hidden test set and 77.2% on the development set, improving the exact set match accuracy by 1.5% and 0.8%, respectively, compared with the optimal baseline S²SQL+ELECTRA. Regarding the development and test sets, it can be observed that (1) the exact set match accuracy of RGISQL on both the development and test sets has significantly improved, indicating that RGISQL can generate SQL queries more accurately, demonstrating a clear enhancement in the model's understanding of natural language questions. (2) The model's accuracy on the unseen test set has also increased, suggesting a significant improvement in its generalization ability.

As shown in Table 4, RGISQL+ELECTRA achieved 77.3% on the hidden test set and 82.6% on the development set, improving execution accuracy by 1.8% and 2.1%, respectively, compared with the optimal baseline RASAT+PICARD. The RGISQL model's execution accuracy on both the development and test sets has significantly improved, indicating that RGISQL can not only generate SQL queries more accurately but also produce SQL queries with more precise execution results. This suggests that the SQL queries generated by RGISQL meet the requirements more effectively and are better aligned with practical needs.

We also compared the model's more fine-grained performance at different levels of complexity, as shown in Table 5. The experimental results indicate that RGISQL demonstrates higher performance and better generalization capabilities when handling datasets of varying complexity. Particularly, RGISQL achieved significant improvements over previous methods when handling datasets with complex grammatical structures and questions with diverse meanings, especially those containing long queries. This indicates that RGISQL has a clear advantage in processing complex and lengthy query problems.

Table 5. Results of SQL query generation at different levels of complexity.

Approach	Easy	Medium	Hard	Extra Hard
LGESQL [8]	86.3	69.5	61.5	41.0
LGESQL+ELECTRA [8]	91.5	76.7	66.7	48.8
RGISQL	86.5	69.8	63.0	42.3
RGISQL+ELECTRA	93.0	77.0	67.7	50.5

Additionally, as shown in Table 6, RGISQL also demonstrates significant improvements in robustness on the dataset Spider-Syn, indicating that the SQL queries generated by RGISQL exhibit strong robustness. This finding emphasizes our method's ability to handle various, complex real-world situations and highlights its stability and reliability when facing unknown or changing data, suggesting an enhancement in the model's generalization capability.

Table 6. The exact set match accuracy of different methods on the Spider-Syn dev and test sets.

Models	Acc
Global-GNN [18]	23.6
IRNet [24]	28.4
RATSQL+Grappa [7]	49.1
S ² SQL+Grappa [30]	51.4
RGISQL+Grappa	52.6

5.5. Ablation Experiments

RGISQL consists of four major components: refined grammatical information, reduced coupling, segmentation processing, and the RAT layer. To assess the influence of these components on model performance, each component was removed or replaced one at a time, and the performance was compared with that of the original model. The ablation experiments were conducted on the development set.

Through ablation experiments on Spider, as shown in Figure 10, our results indicate that the proposed method outperformed traditional methods in several aspects. Firstly, removing the refined grammatical information led to a significant drop in performance, highlighting the importance of detailed syntactic analysis in understanding natural language questions and generating accurate SQL statements. This advantage enables our model to perform better when dealing with complex queries.

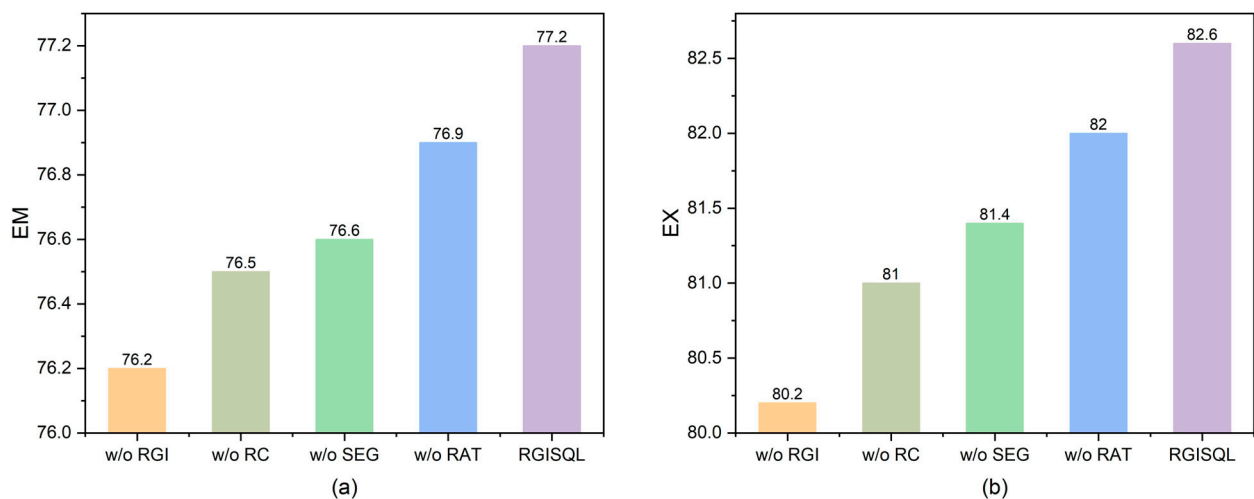


Figure 10. Ablation results on Spider. Among them, (a) represents the results of exact set matching accuracy, and (b) represents the results of execution accuracy. “RGI” stands for refined grammatical information. “RC” stands for reduced coupling. “SEG” refers to segmentation processing. “EM” refers to exact set match accuracy. “EX” refers to execution accuracy.

Secondly, removing components designed to reduce the coupling of edge embeddings also resulted in a decline in performance, indicating that our strategy effectively simplifies the complexity of edge embeddings in graph neural networks, thereby enhancing learning efficiency. The effectiveness of the segmentation strategy for long queries has been validated, and compared with other methods, our model demonstrates superior robustness and generalization capabilities.

Lastly, removing components related to the RAT layers also led to a drop in performance, emphasizing the key role of unifying node and edge embedding representations in enhancing overall performance.

The experiments were also conducted on Spider-Syn, as shown in Figure 11. RGISQL demonstrated significant advantages in the task of text-to-SQL conversion, particularly in its ability to handle complex and lengthy queries, thereby providing a more effective solution in this field.

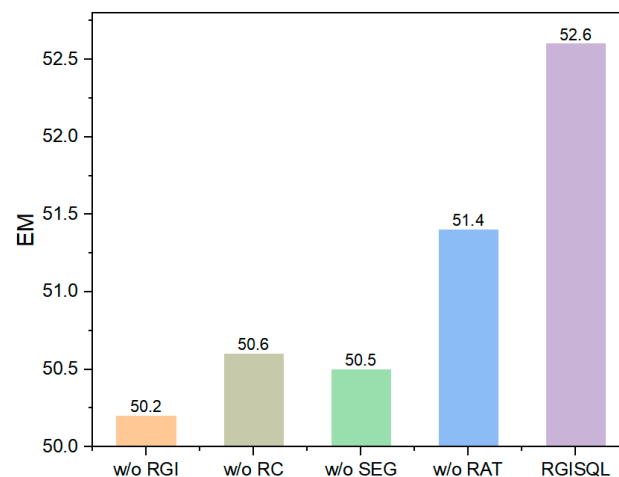


Figure 11. Ablation results on Spider-Syn. “RGI” stands for refined grammatical information. “RC” stands for reduced coupling. “SEG” refers to segmentation processing. “EM” refers to exact set match accuracy.

Additionally, to explore the impact of embedding methods generated by pre-trained language models on various models, RGISQL was compared with two baseline models that utilize RoBERTa. The results are presented in Table 7. It is evident that regardless of the model used to generate embeddings, RGISQL’s performance was significantly superior to that of the other models. This finding indicates that our method does not rely solely on the characteristics of a single model but demonstrates broad adaptability and robustness.

Table 7. Different models’ exact set match accuracy on the development set using RoBERTa pre-trained model.

Models	Dev
RAT+RoBERTa	69.7
S ² SQL+RoBERTa	71.4
RGISQL+RoBERTa	72.0

In Spider, we conducted ablation experiments to evaluate the use of ASTormer and the approach of combining node-centric and edge-centric graphs to update embeddings, with the results shown in Figure 12. The experiments demonstrate that combining these two graph structures effectively enhances the model’s ability to handle complex relationships, resulting in more accurate SQL queries.

The ASTormer decoder enhances SQL generation accuracy by effectively capturing the dependencies between nodes and edges. This multi-relational graph structure demonstrates greater robustness compared with traditional methods, particularly in processing complex natural language queries, where the model excels.

Furthermore, the correlation between different words in natural language questions was calculated. As shown in Figure 13, after incorporating more detailed grammatical information, the correlation among words with added grammatical relationships was significantly higher than that of other words, with the correlation of the four refined grammatical relationships reaching their peak. This indicates that RGISQL is capable of developing a deeper understanding of the grammar present in natural language questions, thereby enhancing its ability to generate more accurate SQL queries and improving overall model performance.

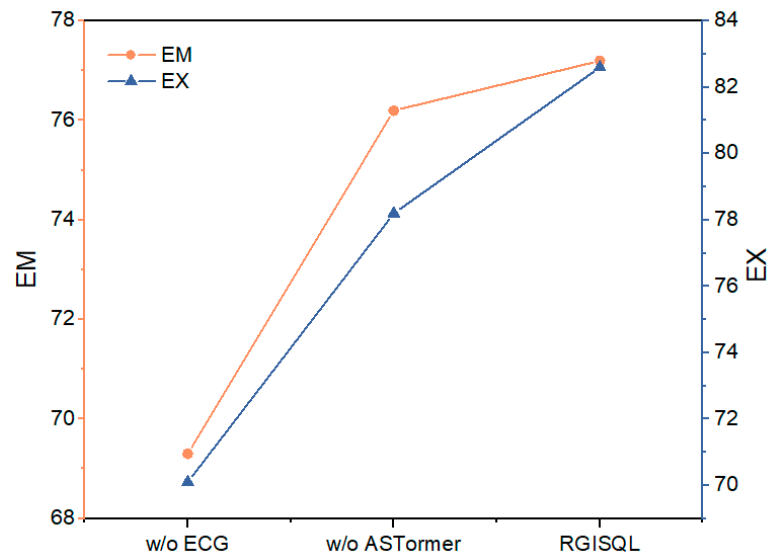


Figure 12. Ablation experiments on the settings of RGAT and the decoder. “ECG” represents the edge-centric graph. “EM” refers to exact set match accuracy.

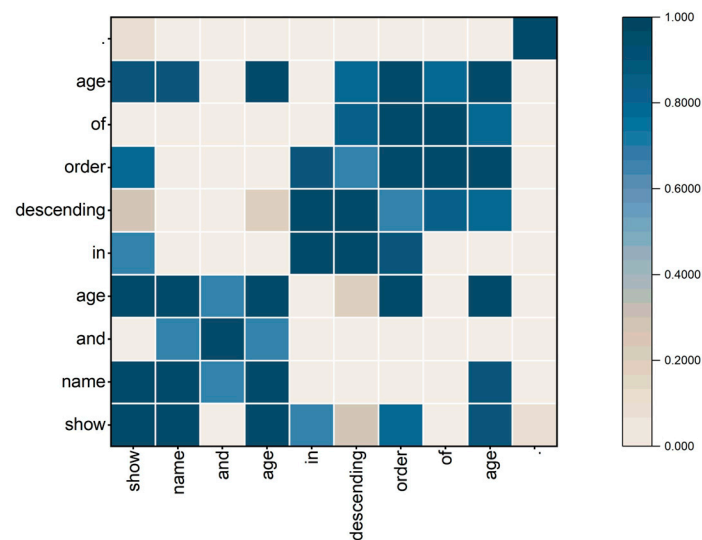


Figure 13. The correlation between different words in natural language questions.

6. Conclusions

In this paper, we propose RGISQL, an advanced method for the text-to-SQL task. RGISQL effectively extracts grammatical dependencies from natural language questions by utilizing grammatical dependency trees, resulting in a more accurate semantic understanding of these questions. To tackle the complexity of long queries, we employ a segmentation approach that enhances the model’s ability to process lengthy inputs. This method integrates various relational structures and refined grammatical dependencies from natural language questions into the RGAT model, while significantly reducing the complexity of edge embeddings through dynamic attention pooling, thereby improving overall model performance. Additionally, recursive decoding is combined with the AS-Tormer decoder for SQL generation, optimizing and refining the generation process to ensure greater accuracy in the resulting outputs. The experimental results demonstrate that RGISQL outperforms baseline methods on the Spider and Spider-Syn datasets, with its effectiveness being validated through extensive ablation studies. RGISQL, proposed in this paper, offers innovative solutions for exploring grammatical dependencies and addressing

complex issues in long queries, which is significant for further improving the accuracy of text-to-SQL generation tasks.

In future work, we aim to process natural language queries more comprehensively by extracting the textual features contained within them, thereby achieving a more precise understanding of semantics and generating more accurate SQL queries. Concurrently, we will optimize the model architecture by adopting more efficient methods to reduce complexity and enhance training speed. Additionally, we plan to integrate existing large language models to further augment the learning capabilities of our model.

Author Contributions: Conceptualization, S.L.; methodology, S.L. and R.Q.; validation, Y.H. and L.A.; writing—original draft preparation, S.L.; writing—review and editing, S.L. and R.Q. All authors have read and agreed to the published version of the manuscript.

Funding: This research study was funded by the Key Research and Development Program of China, grant number 2022YFC3005401, and the Key Technology Project of China Huaneng Group, grant numbers HNKJ20-H46 and HNKJ20-H64.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in the study are included in the article material, and further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Deng, X.; Awadallah, A.H.; Meek, C.; Polozov, O.; Sun, H.; Richardson, M. Structure-Grounded Pretraining for Text-to-SQL. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021.
2. Kamath, A.; Das, R. A Survey on Semantic Parsing. *arXiv* **2018**, arXiv:1812.00978.
3. Yu, T.; Zhang, R.; Yang, K.; Yasunaga, M.; Wang, D.X.; Li, Z.F.; Ma, J.; Li, I.; Yao, Q.N.; Roman, S.; et al. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018.
4. Gan, Y.J.; Chen, X.Y.; Huang, Q.P.; Purver, M.; John, W.; Xie, J.X.; Huang, P.S. Towards Robustness of Text-to-SQL Models against Synonym Substitution. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Online, 1–6 August 2021.
5. Scarselli, F.; Gori, M.; Tsoi, A.C.; Hagenbuchner, M.; Monfardini, G. The Graph Neural Network Model. *IEEE Trans. Neural Netw.* **2009**, *20*, 61–80. [[CrossRef](#)] [[PubMed](#)]
6. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; Berg, R.V.D.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the 15th International Semantic Web Conference, Heraklion, Greece, 3–7 June 2018.
7. Wang, B.L.; Shin, R.; Liu, X.D.; Polozov, O.; Richardson, M. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2021.
8. Cao, R.S.; Chen, L.; Chen, Z.; Zhao, Y.B.; Zhu, S.; Yu, K. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and NonLocal Relations. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, Online, 1–6 August 2021.
9. Qi, J.X.; Tang, J.Y.; He, Z.W.; Wan, X.P.; Cheng, Y.; Zhou, C.H.; Wang, X.B.; Zhang, Q.S.; Lin, Z.H. RASAT: Integrating Relational Structures into Pretrained Seq2Seq Model for Text-to-SQL. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Abu Dhabi, United Arab Emirates, 7–11 December 2022.
10. Zelle, J.M.; Mooney, R.J. Learning to parse database queries using inductive logic programming. In Proceedings of the National Conference on Artificial Intelligence, Philadelphia, PA, USA, 4–9 August 1996.
11. Zettlemoyer, L.S.; Collins, M. Learning to map sentences to logical form: Structured classification with probabilistic categorical grammars. In Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence, Arlington, VA, USA, 22–26 July 2005.
12. Xu, X.J.; Liu, C.; Song, D. SQLNet: Generating Structured Queries from Natural Language Without Reinforcement Learning. *arXiv* **2017**, arXiv:1711.04436.
13. Yu, T.; Yasunaga, M.; Yang, K.; Zhang, R.; Wang, D.X.; Li, Z.F.; Radev, D. SyntaxSQLNet: Syntax Tree Networks for Complex and Cross-Domain Text-to-SQL Task. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018.

14. Choi, D.H.; Shin, M.C.; Kim, E.G.; Shin, D.R. *RYANSQL: Recursively Applying Sketch-Based Slot Fillings for Complex Text-to-SQL in Cross-Domain Databases*; Computational Linguistics: Cambridge, MA, USA, 2021; pp. 309–332.
15. Hui, B.Y.; Shi, X.; Geng, R.Y.; Li, B.H.; Li, Y.B.; Sun, J.; Zhu, X.D. Improving Text-to-SQL with Schema Dependency Learning. *arXiv* **2021**, arXiv:2103.04399.
16. Kelkar, A.; Relan, R.; Bhardwaj, V.; Vaichal, S.; Relan, P.A. Bertrand-DR: Improving Text-to-SQL using a Discriminative Reranker. *arXiv* **2020**, arXiv:2002.00557.
17. Rubin, O.; Berant, J. SmBoP: Semi-autoregressive Bottomup Semantic Parsing. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021.
18. Bogin, B.; Berant, J.; Gardner, M. Representing Schema Structure with Graph Neural Networks for Text-to-SQL Parsing. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019.
19. Scholak, T.; Li, R.; Bahdanau, D.; Vries, H.D.; Pal, C. DuoRAT: Towards Simpler Text-to-SQL Models. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021.
20. Chen, Z.; Chen, L.; Zhao, Y.B.; Cao, R.S.; Xu, Z.H.; Zhu, S.; Yu, K. ShadowGNN: Graph Projection Neural Network for Text-to-SQL Parser. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021.
21. Veličković, P.; Cucurull, G.; Casanova, A.; Romero, A.; Lio, P.; Bengio, Y.S. Graph attention networks. *arXiv* **2017**, arXiv:1710.10903.
22. Shaw, P.; Uszkoreit, J.; Vaswani, A. Self-Attention with Relative Position Representations. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, New Orleans, LO, USA, 1–6 June 2018.
23. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, Red Hook, NY, USA, 4–9 December 2017.
24. Guo, J.Q.; Zhan, Z.C.; Gao, Y.; Xiao, Y.; Lou, J.G.; Liu, T.; Zhang, D.M. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019.
25. Lei, W.Q.; Wang, W.X.; Ma, Z.X.; Gan, T.; Lu, W.; Kan, M.Y.; Chua, T.S. Re-examining the Role of Schema Linking in Text-to-SQL. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020.
26. Ishiwatari, T.C.; Yasuda, Y.; Miyazaki, T.; Goto, J. Relation-aware Graph Attention Networks with Relational Position Encodings for Emotion Recognition in Conversations. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020.
27. Wang, Y.; Sun, Y.B.; Liu, Z.W.; Sarma, S.E.; Bronstein, M.; Solomon, J.M. *Dynamic Graph CNN for Learning on Point Clouds*; Association for Computing Machinery: New York, NY, USA, 2019; pp. 1–12.
28. Cao, R.; Zhang, H.; Xu, H. ASTormer: An AST Structure-aware Transformer Decoder for Text-to-SQL. *arXiv* **2023**, arXiv:2310.18662.
29. Peng, Q.; Zhang, Y.H.; Zhang, Y.H.; Bolton, J.; Manning, C.D. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations, Online, 5–10 July 2020.
30. Clark, K.; Luong, M.T.; Le, Q.V.; Manning, C.D. Electra: Pre-training text encoders as discriminators rather than generators. In Proceedings of the International Conference on Learning Representations. International Conference on Learning Representations, Online, 26 April–1 May 2020.
31. Hui, B.Y.; Geng, R.Y.; Wang, L.H.; Qin, B.W.; Li, Y.Y.; Li, B.W.; Sun, J.; Li, Y.B. S2SQL: Injecting Syntax to Question-Schema Interaction Graph Encoder for Text-to-SQL Parsers. In Proceedings of the Association for Computational Linguistics: ACL 2022, Dublin, Ireland, 22–27 May 2022.
32. Lin, X.V.; Socher, R.; Xiong, C.M. Bridging textual and tabular data for crossdomain text-to-SQL semantic parsing. In Proceedings of the Association for Computational Linguistics, Online, 5–10 July 2020.
33. Huang, J.; Wang, Y.; Wang, Y.; Dong, Y.; Xiao, Y. Relation Aware Semi-autoregressive Semantic Parsing for NL2SQL. *arXiv* **2021**, arXiv:2108.00804.
34. Gan, Y.J.; Chen, X.Y.; Xie, J.X.; Purver, M.; Woodward, J.R.; Drake, J.; Zhang, Q.F. Natural SQL: Making SQL easier to infer from natural language specifications. In Proceedings of the Association for Computational Linguistics, Punta Cana, Dominican Republic, 6–11 June 2021.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.